

Scala-Cheatsheet

Syntaktische Konstrukte

Name	VL	Beschreibung	Beispiel
Listengeneratoren	10	Generiert eine Liste. Die Liste enthält für alle Werte im kartesischen Produkt der Generatoren, für die der Wächter erfüllt ist, den durch <code>yield</code> angegebenen Ausdruck.	<code>for (x <- 0 to 10 if (x % 2 == 0)) yield x</code>
Mustervergleich	10	Vergleicht eine angegebene Instanz mit vorgegebenen Mustern. Die Anweisungen des ersten passenden Musters (von oben aus betrachtet) werden ausgeführt. Zusätzliche Angabe eines Wächters (guard) möglich.	<code>x match { case Nil => 0 case h::ts if (h < 0) => 4 case h::ts => 7 }</code>
Sektionen	10	Kurzschreibweise für (i. d. R. binäre) Infix-Operatoren.	<code>_ + 1</code>
Faule Auswertung	11	Wertet die rechte Seite der Definition nur aus, sobald es zwingend für die Programmausführung notwendig ist.	<code>lazy val x = List.range(1, 1000).product</code>
Algebraische Datentypen	11	Datentypen bzw. Objekte, die für Mustervergleich verwendet werden können.	<code>abstract class Tree case object Empty extends Tree case class Node(l: Tree, v: Int, r: Tree) extends Tree</code>
Generische Definitionen	10/11	Typparameter werden mit eckigen Klammern, Kovarianz durch ein <code>+</code> angezeigt.	<code>class C[+T](value: T)</code>
Partielle Fkt.-Auswertung	11	Funktionen mit mehreren Parameterlisten können als Funktionen, die andere Funktionen zurückgeben, betrachtet werden.	<code>def f: Int => Int => Int = a => b => a-b val g = f(10) g(6) == 4</code>

Tupel

Name	VL	Beschreibung	Beispiel
<code>(x, ...)</code>	10	Erzeugt ein Tupel bestehend aus den angegebenen Komponenten (maximal 22 Komponenten).	<code>(4, 5)</code>
<code>_1, _2, ...</code>	10	Liefert die n -te Komponente eines Tupels.	<code>(4, 5, 6)._2 == 5</code>

Listenfunktionen

Name	VL	Beschreibung	Beispiel
<code>range</code>	10	Erzeugt eine Liste mit allen Elementen zwischen der unteren (einschließlich) und der oberen (ausschließlich) Grenze.	<code>List.range(4, 7) == List(4, 5, 6)</code>

Listenmethoden

Name	VL	Beschreibung	Beispiel
<code>::</code>	10	Liefert eine Liste bestehend aus einem Kopfelement und einer Restliste.	<code>4 :: List(5, 6) == List(4, 5, 6)</code>
<code>:::</code>	10	Liefert eine Liste, bei der linke und rechte Liste zusammengehängt wurden.	<code>List(4, 5) ::: List(5, 6) == List(4, 5, 5, 6)</code>
<code>apply</code>	10	Liefert das n -te Element einer Liste.	<code>List(4, 5, 6)(2) == 6</code>
<code>head</code>	10	Liefert das erste Element einer Liste, sofern vorhanden.	<code>List(4, 5, 6).head == 4</code>
<code>tail</code>	10	Liefert die Elemente einer Liste ohne das erste Element.	<code>List(4, 5, 6).tail == List(5, 6)</code>
<code>last</code>	12	Liefert das letzte Element einer Liste, sofern vorhanden.	<code>List(4, 5, 6).last == 6</code>
<code>take</code>	10	Liefert die ersten n Elemente einer Liste.	<code>List(4, 5, 6).take(2) == List(4, 5)</code>
<code>drop</code>	10	Liefert eine Liste ohne die ersten n Elemente.	<code>List(4, 5, 6).drop(2) == List(6)</code>
<code>length</code>	10	Liefert die Länge einer Liste.	<code>List(5, 6, 7).length == 3</code>
<code>sum/product</code>	10	Liefert die Summe/das Produkt der Elemente einer Liste.	<code>List(4, 5, 6).sum == 15</code>
<code>reverse</code>	10	Liefert eine Liste, deren Elemente in umgekehrter Reihenfolge stehen.	<code>List(4, 5, 6).reverse == List(6, 5, 4)</code>
<code>zip</code>	10	Fügt die Elemente zweier Listen paarweise zusammen.	<code>List(4, 5, 6).zip(List('A', 'B')) == List((4, 'A'), (5, 'B'))</code>
<code>filter</code>	10, 11	Liefert die Elemente einer Liste, die die gegebene Bedingung erfüllen.	<code>List(4, 5, 6).filter(_%2 == 0) == List(4, 6)</code>

map	11	Wendet die gegebene Funktion auf jedes Element der Liste an.	List(4, 5, 6).map(_+1) == List(5, 6, 7)
takeWhile	11	Wählt Elemente aus einer Liste aus, solange die gegebene Funktion zutrifft.	List(4, 5, 6).takeWhile(_%2==0) == List(4)
dropWhile	11	Wählt diejenigen Elemente aus einer Liste aus, die takeWhile nicht zurückliefern würde.	List(4, 5, 6).dropWhile(_%2==0) == List(5, 6)
forall	11	Prüft, ob die gegebene Bedingung für alle Elemente zutrifft.	List(4, 5, 6).forall(_%2==0) == false
exists	11	Prüft, ob die gegebene Bedingung für mindestens ein Element zutrifft.	List(4, 5, 6).exists(_%2==0) == true
foldRight	11	Wendet eine Reduktion von rechts nach links auf die Elemente einer Liste an. Für einen Startwert s , eine Funktion f und Listenelemente $x_1, \dots, x_n$ wird also $f(x_1, f(\dots f(x_n, s)))$ berechnet.	List(4, 5, 6).foldRight(0)(_ - _) == 5
foldLeft	11	Wie foldRight, aber von links nach rechts. Für einen Startwert s , eine Funktion f und Listenelemente $x_1, \dots, x_n$ wird also $f(f(\dots f(s, x_1), \dots), x_n)$ berechnet.	List(4, 5, 6).foldLeft(0)(_ - _) == -15
min/max	12	Wählt das minimale/maximale Element einer Liste aus.	List(5, 4, 6, 5).min == 4
flatMap	13	Wie map, allerdings muss die gegebene Funktion eine Sammlung von Elementen (z. B. eine Liste) liefern. Diese Listen werden dann verkettet.	List(4, 5, 6).flatMap(x => (1 to x-1).filter(x%_==0)) == List(1, 2, 1, 1, 2, 3)

LazyList-Funktionen

Name	VL	Beschreibung	Beispiel
iterate	11	Erzeugt eine unendliche LazyList durch wiederholte Anwendung einer Funktion beginnend bei einem Startwert.	LazyList.iterate(4)(_ + 1).take(3).toList == List(4, 5, 6)

LazyList-Methoden

Name	VL	Beschreibung	Beispiel
#::	11	Liefert eine LazyList bestehend aus einem Kopfelement und einer Restliste.	4 #:: LazyList(5, 6) == LazyList(4, 5, 6)
#:::	11	Liefert eine LazyList, bei der linke und rechte Liste zusammengehängt wurden.	LazyList(4,5) #::: LazyList(5,6) == LazyList(4, 5, 5, 6)
apply head tail take drop zip filter map takeWhile dropWhile flatMap		Siehe Listenmethoden.	
length sum product forall exists foldRight foldLeft min max		Für endliche LazyList wie die entsprechenden Listenmethoden. Für unendliche LazyList terminiert die Auswertung nicht (oder nicht notwendigerweise).	
toList	11	Wandelt endliche LazyList in Liste um (terminiert nicht bei unendlichen LazyLists).	LazyList(4, 5, 6).toList == List(4, 5, 6)

Parallelisierung

Name	VL	Beschreibung	Beispiel
par	12	Gibt eine Sicht auf die Daten des entsprechenden Container-Objekts zurück, deren Operationen unter Umständen parallelisiert sind.	List.range(0, 1000).par.map(_ * 2).toList
Future	12	Startet eine nebenläufige Berechnung.	val fut = Future { foo() }
Await.result	12	Wartet auf Beendigung einer nebenläufigen Berechnung.	Await.result(fut, Duration.Inf)